

Inverse Free-form Deformation for interactive UV map editing

Seung-Tak Noh*

seungtak.noh@gmail.com
The University of Tokyo
Tokyo, Japan

Takeo Igarashi†

takeo@acm.org
The University of Tokyo
Tokyo, Japan

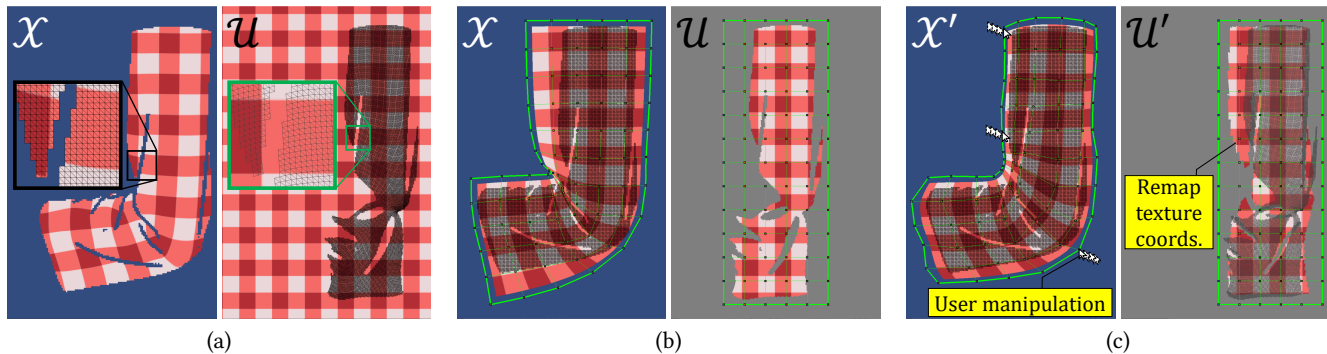


Figure 1: Workflow of our proposed method. (a) We receive a dense image-to-texture mapping as an input. (b) Inverse FFD computes the texture-to-image FFD mapping by least-squares optimization. (c) User can interactively edit the texture mapping via forward FFD in the image space.

ABSTRACT

Free-form deformation (FFD) is useful for manual 2D texture mapping in a 2D domain. The user first places a coarse regular grid in the texture space, and then adjusts the positions of the grid points in the image space. In this paper, we consider the *inverse* way of this problem, namely, *inverse FFD*. In this problem setting, we assume that an initial image-to-texture dense mapping is already obtained by some automatic method, such as a data-driven inference. However, this initial dense mapping may not be satisfactory, so the user may want to modify it. Nonetheless, it is difficult to manually edit the dense mapping due to its huge degrees of freedom. We thus convert the dense mapping to a coarse FFD mapping to facilitate manual editing of the mapping. Inverse FFD is formulated as a least-squares optimization, so one can solve it very efficiently.

CCS CONCEPTS

• Computing methodologies → Texturing.

KEYWORDS

Free-form deformation, least-squares, UV editing

ACM Reference Format:

Seung-Tak Noh and Takeo Igarashi. 2021. Inverse Free-form Deformation for interactive UV map editing. In *SIGGRAPH Asia 2021 Technical Communications (SA '21 Technical Communications)*, December 14–17, 2021, Tokyo, Japan. ACM, New York, NY, USA, 4 pages. <https://doi.org/10.1145/3478512.3488614>

1 INTRODUCTION

Free-form deformation is useful for manual 2D texture mapping in a 2D domain. The user first places a coarse regular grid in the texture space, and then adjusts the positions of the grid points in the image space. This approach has useful applications, such as texturing and replacing the garment in a photograph or illustration (e.g. Adobe Photoshop Warp Grid).

In this paper, we consider the *inverse* way of this problem, namely, *inverse FFD*. In this problem setting, we assume that an initial dense mapping is already computed by some automatic method, such as a data-driven inference [Hashimoto et al. 2020]. We then convert the dense mapping to coarse FFD grid, so that the user can modify the mapping manually. Once FFD mapping is obtained, user can intuitively edit texture mapping by manipulating the FFD control points in the image space.

Inverse FFD is formulated as a least-squares optimization. We clarify that this formulation is linear, so one can solve it very efficiently with an off-the-shelf sparse linear solver. Our proposed system is effective for editing dense-mapped UVs in a way familiar to artists. We demonstrate its effectiveness by showing several editing examples with its computational performances.

Our contributions are as follows:

- A novel inverse free-form deformation method that estimates the FFD grid points from UVs of vertices.
- An interactive system that enables user to edit UVs with simple user interactions.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

SA '21 Technical Communications, December 14–17, 2021, Tokyo, Japan

© 2021 Association for Computing Machinery.

ACM ISBN 978-1-4503-9073-6/21/12...\$15.00

<https://doi.org/10.1145/3478512.3488614>

2 RELATED WORK

FFD is a well-established technique that enables to deform mesh with several control points [Sederberg and Parry 1986]. Because of its generality, it can be applicable for medical images as well [Rueckert et al. 1999]. Standard approach is forward FFD, where the user manipulates the coarse FFD grid, and then the system computes dense mapping. However, we solve the problem of taking a given dense mapping as input and computes corresponding coarse FFD grid; this is why our method is called *inverse* FFD. Hsu et al. [Hsu et al. 1992] used least-squares to minimize *change* to the control points to achieve the given (sparse) *change* to the shape. In contrast, we use least-squares to minimize the difference between the given (dense) shape and the deformation result. Although FFD can support arbitrary grid as control lattice [MacCracken and Joy 1996], it is usually based on the quadrilateral or cubic grid as control lattice. In this research, we suppose quadrilateral grid as target lattice for simplicity.

Texture mapping on 2D image [Corrêa et al. 1998] is very useful for exploring variations of image. Recently, Hashimoto et al. [Hashimoto et al. 2020] presented a data-driven inference method to estimate dense UV mapping for 2D garment line drawing. This initial dense mapping is often unsatisfactory and manual control is necessary. However, it is difficult to directly edit the dense mapping due to high degrees of freedom. Our method mainly aims to make manual post processing easier by using FFD for this kind of texture mapping methods. In our experiment, we used Hashimoto et al. [Hashimoto et al. 2020] method for input meshes, but any methods can be used.

3 THE PROPOSED METHOD

Our proposed method takes a mesh that has unsatisfactory UVs as input. First, we put regular grid in texture space by computing and subdividing the bounding box of input UVs (Figure 1a). We then transfer the control points in the grid from texture space $\mathcal{U}$ to image space $\mathcal{X}$ by solving the least-squares problem (Figure 1b). Once the initial grid are obtained, user can interactively edit UVs by moving those control points in the image space (Figure 1c). Based on the modified grid points, we remap the UVs of the original vertices (Figure 1d).

3.1 Dense image-to-texture mapping as input

The input is given as a mapping from dense regular triangular mesh in image space to deformed triangular mesh in the texture space (Figure 1a left). In this representation, one pixel in the image becomes two triangles to construct triangular mesh. Each vertex is freely mapped to the texture space, so it is non-regularly distributed (Figure 1a right). Because vertices are already associated with its UV coordinate, so rendering is done by simple texture mapping.

3.2 Placement of regular grid in texture space

We first compute a bounding box of the input dense mesh's UV coordinates. We set a small marginal space on the bounding box to ensure to include all vertices in a input mesh. In our prototype, we extend the bounding box by the 5% in both horizontal and vertical directions.

After determining the bounding box, we subdivide this box to obtain the initial grid in the texture space. Although the grid is not necessarily to be quadrilateral [MacCracken and Joy 1996], we discretize UV area as regular grid for simplicity. We subdivide both horizontal and vertical directions and create two-dimensional grid with intersection points. The user can set the number of grid subdivision.

3.3 Inverse FFD

We optimize the position of the control points of the FFD lattice so that the result of FFD mapping is as close as possible to the result of dense input mapping. This is defined as a least-squares problem as follows:

$$\arg \min_P \left(\sum_k^K |\mathbf{x}_k - X(\mathbf{u}_k, P)|^2 + \lambda \sum_i^N \sum_j^M |L_{i,j}(P)|^2 \right) \quad (1)$$

where P is the positions of control points in image space, $\mathbf{x}_k$ is the position of the dense mesh vertices in the image space, $\mathbf{u}_k$ is the position of the dense mesh vertices in the texture space.

The first term is a barycentric energy that reflects the relative position of a vertex in a specific cell to its four control points. More specifically, we locate the cell index (i, j) in the grid, and compute barycentric weights $w_{i,j} \in [0.0 : 1.0]$:

$$X(\mathbf{u}_k, P) = \begin{bmatrix} \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots \\ \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots \\ \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots \\ \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots \\ \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots \\ \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots \\ \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots \end{bmatrix} \begin{pmatrix} P_{i,j} \\ P_{i+1,j} \\ \vdots \\ P_{i,j+1} \\ P_{i+1,j+1} \\ \vdots \\ \vdots \end{pmatrix} \quad (2)$$

Although this formulation simplifies 3rd order B-spline free-form deformation [Rueckert et al. 1999] as 1st order, we did not observe any severe issues in our experiment.

The first term alone is not sufficient because some control points are under constrained (for instance, Figure 2a). Therefore, we add a regularization term $L(\cdot)$ that tries to consecutive horizontal and vertical edges equal:

$$\begin{aligned} \sum ((V_{i,j} - V_{i+1,j}) - (V_{i-1,j} - V_{i,j}))^2 & \quad (\text{horizontal}), \\ \sum ((V_{i,j} - V_{i,j+1}) - (V_{i,j-1} - V_{i,j}))^2 & \quad (\text{vertical}). \end{aligned}$$

In matrix form,

$$L_{i,j}(P) = \begin{bmatrix} \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots \\ \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots \\ \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots \\ \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots \\ \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots \\ \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots \\ \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots \end{bmatrix} \begin{pmatrix} P_{i,j-1} \\ P_{i-1,j} \\ P_{i,j} \\ P_{i+1,j} \\ \vdots \\ P_{i,j+1} \\ \vdots \end{pmatrix} \quad (3)$$

Because both terms are linear in P , we can easily compute the optimum P by solving a sparse linear least-squares problem.

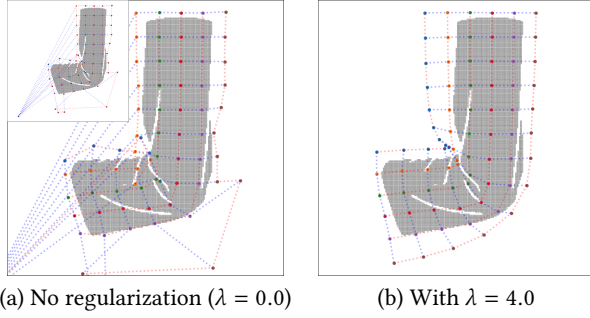


Figure 2: The impact of regularization term.

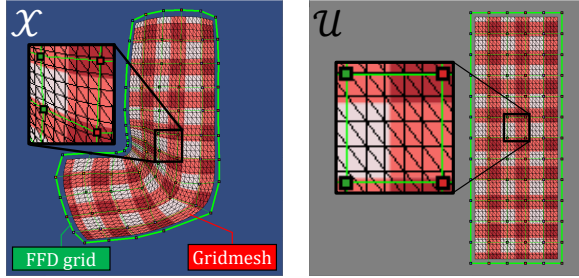


Figure 3: Visualization of manually edited FFD.

To solve the Equation 1 with an off-the-shelf one-dimensional sparse linear solver, we rearrange two-dimensional vectors as one-dimension. With this rearrangement, $\mathbf{x}$ and $\mathbf{P}$ becomes one dimensional vectors, and matrix becomes $([K+(N-2) \cdot M+N \cdot (M-2)] \times 2) \times ([N \cdot M] \times 2)$. We used LSMR [Fong and Saunders 2011] to solve the Equation 1 in our prototype, but any least-squares solver can be used.

3.4 UV editing

After having an initial FFD mapping from the texture space the image space as a result of inverse FFD, the user can modify the FFD mapping by moving the control points of FFD mapping in the image space. To visualize the mapping result in real-time, we place dense regular triangle mesh in the texture space, map the mesh to the image space by *forward* FFD with cubic B-spline [Rueckert et al. 1999], and render the mesh in the image space with texture mapping. In our prototype, we generate $32 (= 4 \times 4 \times 2)$ triangles per grid cell with $25 (= 5 \times 5)$ vertices.

Once editing is confirmed, we remap UVs of gridmesh to the original mesh. Since naïve brute force matching takes so much time, we utilize k-d tree structure for fast computation. We first build k-d tree using gridmesh vertices, and then query each vertex in original mesh. By referring triangles associated with nearest vertices, we find the corresponded gridmesh triangle for the vertex in original mesh efficiently.

4 RESULT AND DISCUSSIONS

We implemented our prototype in Python 3 with common scientific library, such as numpy and scipy. For experimental data, we

Table 1: Performance evaluation for our inverse FFD. (a) constructing sparse matrix, (b) solving least squares, (c) computing FFD for gridmesh in image space, and (d) remapping UVs based on the gridmesh. All times are given in seconds. Images are provided in Figure 1 and Figure 5.

Example	#verts	CPs	(a)	(b)	(c)	(d)
Long sleeve	11,915	6×13	1.070	0.045	0.067	6.117
T-shirt body	14,285	6×8	1.285	0.048	0.043	7.298
— left sleeve	1,103	6×8	0.113	0.008	0.044	0.466
— right sleeve	3,308	6×8	0.294	0.015	0.044	1.480
Cloak	19,450	10×12	2.126	0.254	0.104	9.851
Scarf top	7,144	10×8	0.648	0.048	0.074	3.599
— middle	9,675	8×10	0.864	0.050	0.074	4.963
— bottom	2,270	6×6	0.214	0.009	0.032	1.133

prepared 2D meshes with UV coordinates using the method by Hashimoto et al. [Hashimoto et al. 2020]. Those images are shown in Figure 5. We set λ as 4.0 that generally finds a decent result (Figure 2b).

4.1 Computational Performance

For performance evaluation, we used a desktop equipped with an Intel Core i7 3.2 GHz CPU and 32 GB of RAM. Table 1 shows the computation time for each step in our proposed method. In general, our proposed method supports interactive visualization because only FFD for gridmesh (Table 1c) is related to the real-time performance. Constructing sparse matrix and solving least squares (Table 1a and b) took one to two seconds, and these two steps are computed only once to obtain the FFD grid in the initialization. Remapping UVs (Table 1d) took time compared to other steps, but we think that this is acceptable because it happens only once in the end after user interaction.

4.2 Limitations

The accuracy of inverse FFD is largely dependent on the initial UV mapping. In other words, our method is not so helpful when dense mapping is failed. For instance, FFD lattice can be distorted due to the overlapped vertices in texture space that are usually generated

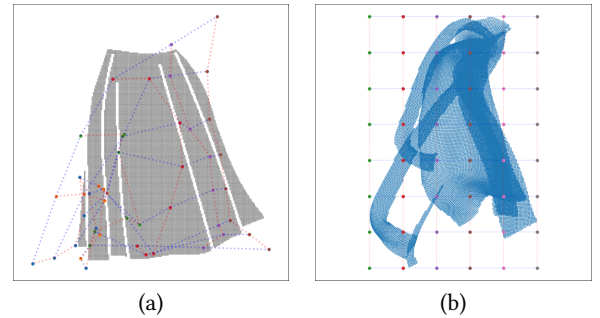


Figure 4: Failure case. Our inverse FFD method cannot compute lattice correctly (a) due to the overlapped vertices in texture space (b).



Figure 5: Our editing results. (a) input mesh, (b) an initial grid by our inverse FFD, (c) modified grid by user, and (d) remapped result without FFD grid for clearance.

by failure of dense mapping (Figure 4). Although we may avoid this problem by carefully selecting submeshes without overlapped UVs, we cannot expect such a temporary expedient in general.

Since our UV editing part is based on FFD, it is difficult to support discontinuity in the texture mapping. Although we support to compute FFD grid with submeshes (e.g. Figure 5), it is not so easy to edit those discontinuities with a free-form deformation by user. Further editing method is needed to support this kind of discontinuity in the texture space.

5 CONCLUSION

We presented a novel inverse FFD method that enables user to modify texture coordinate in the image space. In our approach, we converted the dense mapping to a coarse FFD mapping to facilitate manual editing of the mapping. The problem was formulated as a least-squares optimization, so one can solve it very efficiently. We demonstrated its computational performances and several editing results with FFD lattice. Although our method is dependent on the initial UV mapping, it can provide intuitive and interactive UV editing.

REFERENCES

- Wagner Toledo Corrêa, Robert J. Jensen, Craig E. Thayer, and Adam Finkelstein. 1998. Texture Mapping for Cel Animation. In *Proceedings of the 25th Annual Conference on Computer Graphics and Interactive Techniques (SIGGRAPH '98)*. Association for Computing Machinery, New York, NY, USA, 435–446. <https://doi.org/10.1145/280814.280949>
- David Chin-Lung Fong and Michael Saunders. 2011. LSMR: An Iterative Algorithm for Sparse Least-Squares Problems. *SIAM Journal on Scientific Computing* 33, 5 (2011), 2950–2971. <https://doi.org/10.1137/10079687X>
- Miyu Hashimoto, Tsukasa Fukusato, and Takeo Igarashi. 2020. Neurally-Guided Texturing for Garment Line Drawings. In *SIGGRAPH Asia 2020 Technical Communications (Virtual Event, Republic of Korea) (SA '20)*. Association for Computing Machinery, New York, NY, USA, Article 3, 4 pages. <https://doi.org/10.1145/3410700.3425428>
- William M. Hsu, John F. Hughes, and Henry Kaufman. 1992. Direct Manipulation of Free-Form Deformations. *SIGGRAPH Comput. Graph.* 26, 2 (July 1992), 177–184. <https://doi.org/10.1145/142920.134036>
- Ron MacCracken and Kenneth I. Joy. 1996. Free-Form Deformations with Lattices of Arbitrary Topology. In *Proceedings of the 23rd Annual Conference on Computer Graphics and Interactive Techniques (SIGGRAPH '96)*. Association for Computing Machinery, New York, NY, USA, 181–188. <https://doi.org/10.1145/237170.237247>
- D. Rueckert, L.I. Sonoda, C. Hayes, D.L.G. Hill, M.O. Leach, and D.J. Hawkes. 1999. Nonrigid registration using free-form deformations: application to breast MR images. *IEEE Transactions on Medical Imaging* 18, 8 (1999), 712–721. <https://doi.org/10.1109/42.796284>
- Thomas W. Sederberg and Scott R. Parry. 1986. Free-Form Deformation of Solid Geometric Models. In *Proceedings of the 13th Annual Conference on Computer Graphics and Interactive Techniques (SIGGRAPH '86)*. Association for Computing Machinery, New York, NY, USA, 151–160. <https://doi.org/10.1145/15922.15903>